

Updating the plant PC

How a change made away from the floor gets onto the machine that runs the pouring log, without anybody guessing at which files to copy.

FOR – WHOEVER LOOKS AFTER THE PLANT PC
KEEP WITH THE HANDBOOK · SEPTEMBER 2026

One file, one menu option

An update arrives as a single zip file with a version in its name, like `mes_update_PT-V3.42.zip`.

Everything in this guide is what happens around those five steps, so if you have done this before, the five steps are the whole job.

- Copy the zip into the **updates** folder on the plant PC. Same folder every time. Nothing else goes in there.
- Run **START_HERE.bat** from the project folder.
- Choose **7 — Apply an update**.
- Read what it says it is about to do, then type **YES**. Anything other than YES stops it, and nothing is changed.
- When it finishes, choose **3** to start the app again. Operators pick up the new version the next time their page loads.

IF IT GOES WRONG, IT UNDOES ITSELF

Every step that can fail is checked, and a failure after the files have moved puts the old version back before it tells you anything. You do not have to undo it by hand, and the worst case is that nothing changed.

WHAT IT NEVER TOUCHES

Your database connection, the backups folder, the logs, uploaded photos and the Python environment all belong to that machine rather than to a release. An update never writes to any of them, and a package that tries is refused before anything else happens.

WHO CAN DO THIS

Anybody who can sign into the plant PC and copy a file. There is nothing to know about Python, databases or version control, and nothing to decide. If the machine cannot take the update it says so and changes nothing.

Doing it, and what you see

1 Pick a quiet moment

The app is unavailable for a minute or two while this runs, and an operator who is halfway through typing a log will lose what they typed. Between shifts is ideal. Mid-pour is not.

2 Copy the zip into `updates\`

Straight from the USB stick into that folder. Do not unzip it. If the folder already holds an older package, the newest one is the one it picks up, but it is tidier to keep only the one you are applying.

3 Run **START_HERE.bat** and choose 7

The same menu you use to start the app. Option 7 is *Apply an update*, under the UPDATE heading at the bottom.

4 Check the two lines at the top

It prints the version this PC is running and the name of the package it found, then what the package changes. If either version is not what you expected, type anything other than YES and nothing happens.

5 Type YES and wait

It works through six numbered steps and says what it is doing. A minute is normal. Longer than five means something is stuck, and the safe thing is to leave it rather than close the window.

6 Start the app again

It finishes by printing the version now running. Choose 3 from the menu to start the app, then open it once yourself and sign in before you walk away.

What it is actually doing

Six steps, in this order on purpose. Each one is a thing that used to be a decision somebody made differently every time.

- 1 Reads the package and checks every file**
Each file carries a checksum. A stick that copied half a file looks exactly like a good one until those are read, and that is the failure most likely to actually happen. A bad one stops here, before anything is written.
- 2 Checks the version**
A package built for a different version, one this PC has already had, or an older one going backwards are all refused. Nothing is changed by any of them.
- 3 Takes a database backup**
Into `backups\`, the same as the daily one. If the backup fails it stops there, because an update with no backup behind it is the one that cannot be undone.
- 4 Copies the whole project aside**
Into `rollback\`, in a folder named for the version and the date. That copy is what gets put back if anything later fails, and it is also what you copy back by hand a week later if you need to.
- 5 Writes the files**
Only the files in the package, into the folders they belong in. Files a release removes are removed. Anything that belongs to this machine is left alone.
- 6 Proves it runs**
It compiles every file it copied and then starts the application, which is what applies any database change the release brought with it. If that fails, the old version goes back automatically.

WHERE THINGS END UP

`rollback\` a dated copy of the version you were on · `updates\applied\` packages already used, so one cannot run twice · `logs\updates.log` one line per attempt · `backups\` the database backup from step 3.

When it says something is wrong

No update package found

The zip is not in the `updates` folder, or it is still in a subfolder from unzipping something. Put the zip itself in there.

does not match its checksum

The file did not copy properly. Copy it onto the stick again and copy it across again. Nothing was changed on the PC.

This package updates PT-V3.40, and this PC is on PT-V3.38

You are missing a release in between, or this package was built for a different machine. Apply them in order, or ask for one built against the version this PC is actually on. The version it is on is printed at the top of the screen.

This PC is already on that version

The package has already been applied here. There is a copy of every package that has been used in `updates\applied\`.

the backup failed

The database is not reachable, or PostgreSQL is not running. Choose 5 from the menu to diagnose the PC. Nothing was changed, and the update is worth fixing this first rather than skipping it.

the application did not start

The update did not take, and the old version has already been put back. The PC is running exactly what it was running before. Send me the last twenty lines of what it printed, and the backup from that run is in `backups\` if it is ever needed.

a package could not be installed

The release needed something new from the internet and this PC could not reach it. The update rolled itself back. It needs a package bundle built for it, which is my end, not yours.

Going back, and building one

Putting the old version back by hand

You should not need this. The applier undoes itself when a step fails, and this is for the other case: it applied cleanly and something looks wrong an hour later.

1 Close the app

Close the window that is running it, so nothing is reading the files you are about to replace.

2 Copy the dated folder back over the top

Open `rollback\`, find the folder named for the version you want and the time it was taken, and copy everything inside it into the project folder, replacing what is there.

3 Start it again and check the version

The version is printed at the bottom of the sidebar in the app, and by option 5 on the menu.

THE ONE CASE THAT NEEDS MORE THAN FILES

If the release you are undoing changed the database, putting the old files back leaves older code in front of a newer database. Usually that is fine. If the app misbehaves after a rollback, restore the backup that was taken at the start of that update, which is the newest file in `backups\`. Tell me before you do it, because restoring the database also gives back any logs entered since.

Building a package

My end, written down so it is not only in my head. From the project folder on the machine where the change was made:

```
python dev/make_update.py --from PT-V3.41 --notes "what is in it"
```

It reads what changed since that version and writes one zip into `dist\`, listing what went in it. The version after `--from` is the version the plant PC is running, and getting it wrong is safe: the applier refuses a package that does not match the machine.